Assignment 2

Group 17

Course: CS671 – Deep Learning

Part 1: The Geometry of Convolutions

Goal: Go beyond treating the network as a black box and to develop a clear understanding of how spatial dimensions change across layers.

1 Task

CNN consist of convolutional layers,pooling layers,and fully connected layers. Convolutional layers extract spatial features, while pooling layers reduce spatial dimensions. Fully connected layers perform classification based on extracted features.

Pooling layers play a crucial role in reducing the spatial size of feature maps, which directly affects the number of parameters in the Fully Connected layer.

Our task is to make a CNN with:

- Input image size : $64 \times 64 \times 3$
- 3 Convolution layers of kernel size 3×3, padding=1,stride=1
- 2 MaxPooling layers of kernel size 2×2, stride=2
- 1 Fully connected layer with 10 output neurons

2 Output Size Formula

The output size of a convolution layer is given by:

$$H_{out} = \frac{H_{in} + 2P - K}{S} + 1 \tag{1}$$

where:

- H_{in} = input height

- P = padding
- K = kernel size
- S = stride

Pooling output size:

$$H_{out} = \frac{H_{in} - K}{S} + 1 \tag{2}$$

3 CNN architecture

Layer	no. of nodes
Conv1	16
Conv2	32
Conv3	64
final output layer	10

4 Case Analysis

4.1 Case 1: Both Pooling and padding Layer Present

Step-by-step dimensions:

- Input: $64 \times 64 \times 3$
- Conv1: $64 \times 64 \times 16$
- Pool1: $32 \times 32 \times 16$
- Conv2: $32 \times 32 \times 32$
- Pool2: $16 \times 16 \times 32$
- Conv3: $16 \times 16 \times 64$

Flatten size:

$$16 \times 16 \times 64 = 16384 \tag{3}$$

FC parameters:

$$16384 \times 10 + 10 = 163,850 \tag{4}$$

4.2 Case 2:No padding Present

- Input: $64 \times 64 \times 3$
- Conv1: $62 \times 62 \times 16$
- Pool1: $31 \times 31 \times 16$
- Conv2: $29 \times 29 \times 32$
- Pool2: $14 \times 14 \times 32$
- Conv3: $12 \times 12 \times 64$

Flatten size:

$$12 \times 12 \times 64 = 9216 \tag{5}$$

FC parameters:

$$9216 \times 10 + 10 = 92170 \tag{6}$$

4.3 Case 3: NO Pooling and no padding Present

- Input: $64 \times 64 \times 3$
- Conv1: $62 \times 62 \times 16$
- Conv2: $60 \times 60 \times 32$
- Conv3: $58 \times 58 \times 64$

Flatten size:

$$58 \times 58 \times 64 = 215296 \tag{7}$$

FC parameters:

$$215296 \times 10 + 10 = 2152970 \tag{8}$$

4.4 Case 4: No Pooling Layers with padding

- Input: $64 \times 64 \times 3$
- Conv1: $64 \times 64 \times 16$
- Conv2: $64 \times 64 \times 32$
- Conv3: $64 \times 64 \times 64$

Flatten size:

$$64 \times 64 \times 64 = 262,144 \tag{9}$$

FC parameters:

$$262144 \times 10 + 10 = 2,621,450 \tag{10}$$

5 Comparison Table

Case	Final feature map size	Flatten size	FC parameters
Pooling Layers with padding	$16 \times 16 \times 64$	16,384	163,850
pooling layers+ no padding	$12 \times 12 \times 64$	9216	92170
No pooling + No padding	$58 \times 58 \times 64$	215296	2152970
No Pooling Layers with padding	$64 \times 64 \times 64$	262,144	2,621,450

Table 1: Comparison of Fully Connected Layer Parameters

6 Parameter increase analysis

Parameter change relative to case 1 for:

- Case 2 decrease:

$$163850 - 92170 = 71680 \tag{11}$$

- Case 4 increase:

$$2,621,450 - 163,850 = 2,457,600 \tag{12}$$

here,

$$\frac{2,621,450}{163,850} = 16\times \tag{13}$$

this represents a 16 times increase in parameters.

7 Parameter explosion problem

Parameter explosion refers to the rapid increase in the number of trainable parameters when spatial dimensions are not reduced.

Fully Connected layers connect every input neuron to every output neuron:

$$\text{Parameters} = \text{Input size} \times \text{Output size} \tag{14}$$

Without pooling:

- Spatial size remains large
- Flatten size becomes huge
- Count of parameter grows quadratically

Consequences:

- Increased memory usage
- Slower training
- Higher risk of overfitting
- Increased computational cost

Pooling reduces spatial dimensions due to which it makes CNN computationally efficient.

8 Conclusion

This report shows that pooling layers significantly reduce the number of parameters in a CNN by decreasing the spatial dimensions of feature maps before the Fully Connected layer. Without pooling, the parameter count increases up to 16 times, leading to the parameter explosion problem, which causes higher memory usage, slower training, and increased risk of overfitting.

Padding also affects parameter count indirectly. With padding, spatial dimensions are preserved, resulting in larger feature maps and more parameters in the Fully Connected layer. Removing padding reduces spatial dimensions and due to which the number of parameters decreases, but it can also lead to loss of edge information. Thus, pooling and padding must be carefully used to balance parameter efficiency and feature preservation.

Part 2: The "Why CNN?" Experiment

Goal: To compare FCNN and CNN on original and spatially shifted MNIST images to evaluate their robustness to spatial transformations and to show why convolutional architecture is better for computer vision task.

1. The Task

Two different models were trained on the standard MNIST dataset and evaluated their robustness on a custom "Shifted MNIST" test set, created by translating every test image by 4 pixels to the right.

- **Model A (FCNN):** A standard sequential Fully Connected Neural Network.
- **Model B (CNN):** A Convolutional Neural Network implemented using the keras functional API.

2. Network architectures

- **Model A :** This model is a standard Multi-Layer Perceptron (MLP) and the input image is given as a flattened vector of size(28*28,1) .
  - **Input:** 784 neurons (Flattened 28×28 image).
  - **Hidden Layer 1:** 128 neurons (ReLU activation) .
  - **Hidden Layer 2:** 64 neurons (ReLU) .
  - **Output:** 10 neurons (Softmax activation).
- **Model B :** In this model CNN implemented using the keras functional API to clearly define data flow through the network .

Layer (Type)	**Configuration**	**Output Shape**
Input	$28 \times 28 \times 1$ image	$(28, 28, 1)$
Conv2D	32 filter, 3×3 kernel, ReLU	$(26, 26, 32)$
MaxPooling2D	2×2 pool size	$(13, 13, 32)$
Conv2D	64 filter, 3×3 kernel, ReLU	$(11, 11, 64)$
MaxPooling2D	2×2 pool size	$(5, 5, 64)$
Conv2D	128 filter, 3×3 kernel, ReLU	$(3, 3, 128)$
GlobalAvgPool2D	Averages feature maps	(128)
Dense	64 neurons, ReLU	(64)
Output (Dense)	10 neuron, softmax	(10)

Table 1: Architecture of model B

2. Training details

We trained both the models under the following conditions;

- **Optimizer**: Adam
- **Loss Function**: Sparse categorical crossentropy
- **Epochs**: 5
- **Batch Size**: 64
- **Dataset**: Normal MNIST (training Set)

3. Accuracy comparison

The evaluation results on the Normal vs Shifted test sets are as listed below:

Model architecture	Normal acc.	Shifted acc.	Drop
Model A(FCNN)	97.48%	35.78%	**61.70%**
Model B(CNN-functional)	98.42%	88.85%	**9.57%**

Table 2: Comparision of accuracy of FCCN and CNN on Normal and Shifted MNIST

4. Analysis

Why did the FCNN fail?

The FCNN lacks spatial awareness .It treats the input as a flat vector of 784 isolated pixels.When the pixels were shifted, the active pixels move to different indices.Since the weights are tied to specific pixel positions, the FCNN cannot recognize the same feature or pattern in a new location.

Why did the CNN succeed?

The CNN demonstrated robustness due to three major factors:

1. **Weight sharing:** Unlike the FCNN,the CNN does not learn a weight for every pixel but instead,learns a small set of filters that slide across the entire image (by convolution). Hence,if a filter learns to detect a feature it will always detect the feature regardless of where the feature is present.

2. **Local invariance via pooling:** Standard CNNs use Max Pooling to downsample feature maps.A Max Pooling operation selects the highest activation in a window.If a feature moves by a few number of pixels,it might still remain in the same pooling window,resulting in the exact same output.

3. **Global Average Pooling (GAP):** In model B, the Flatten() layer was replaced with Global Average Pooling(GAP).Unlike Flatten(), which preserves spatial sensitivity,GAP computes the average activation of each feature map.This helps the model to detect the presence of a feature without being too sensitive to the exact location,making it robust to the 4 pixel shift .

Part 3: Getting Features Out and Making Them Easy to Find

3.1 The Filter Gallery

Goal

The goal of this experiment is to look at the kernels of the first convolutional layer of a CNN after it has been trained on MNIST and Tiny-ImageNet-10 to see how well it learns.

Methodology

After the CNN models were trained, the weights of the first convolutional layer were removed. To make it easier to see, each kernel was changed to fit in the range $[0, 1]$.

We viewed the MNIST filters as photos in black and white. The filters for Tiny-ImageNet-10 were shown as RGB images because they have three channels.

MNIST First Layer Filters

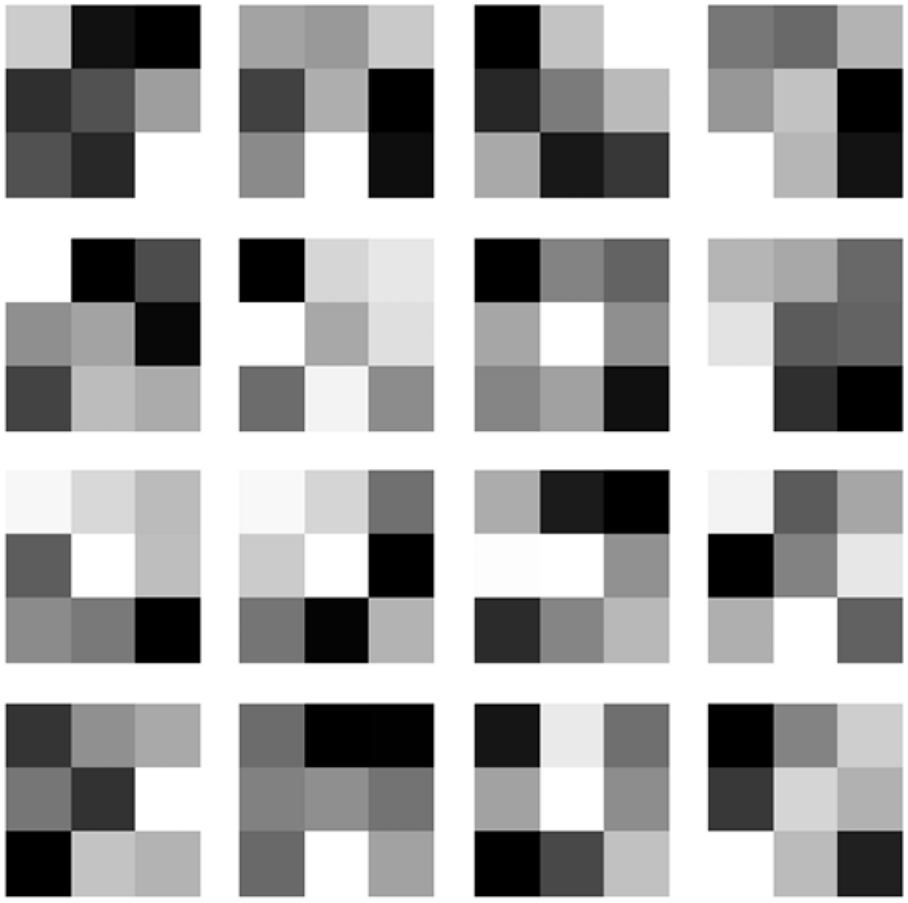


Figure 1: A picture of the first-layer convolutional filters that were learned from MNIST.

Things to look for:

- Some filters look like they can find horizontal and vertical edges.
- Some filters can see diagonal gradients.

- Filters generally look at variations in intensity because MNIST is black and white.

Tiny-ImageNet-10 First Layer Filters

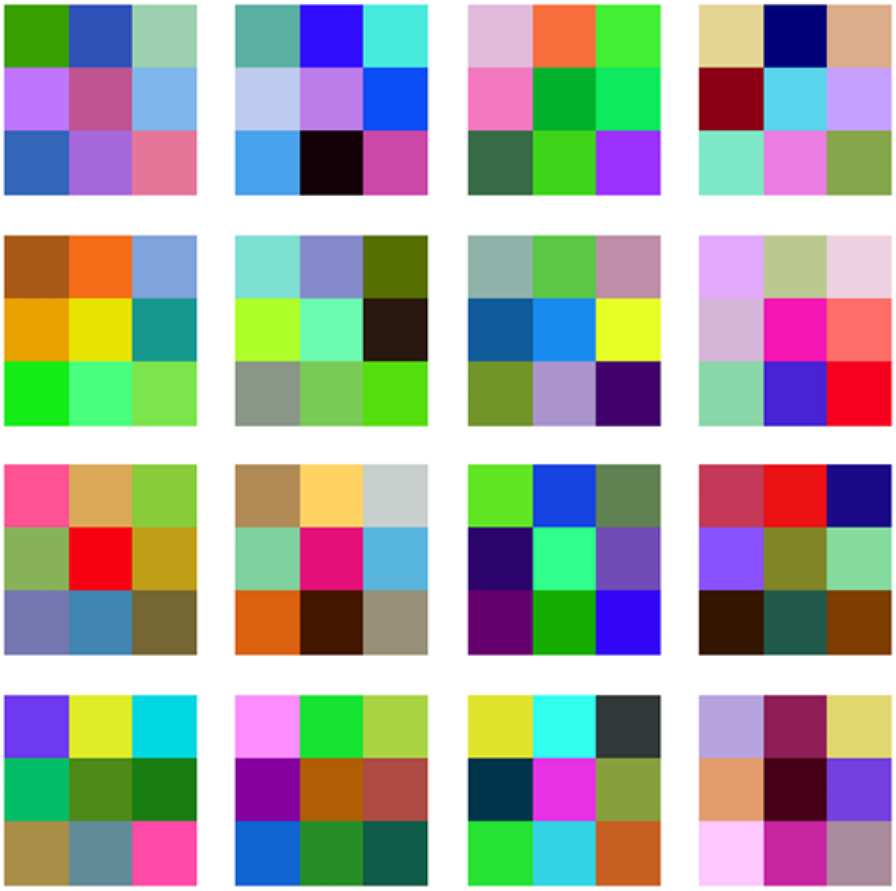


Figure 2: A view of the first-layer convolutional filters that were learned with Tiny-ImageNet-10.

Things to consider about:

- Filters show patterns that are responsive to color because of the RGB channels.
- Some kernels appear like edge detectors that are similar to Gabor's.
- Some filters respond to color blobs and textures that are pointing in a given way.

Comparing with FCNN Weights

CNN filters have a different spatial structure than the Fully Connected Neural Network (FCNN) from Assignment 1.

FCNN weights do not preserve spatial locality, making them non-interpretable in image space. But convolutional filters only learn patterns that are present in certain areas, such edges, gradients, and textures.

Differences between datasets:

- The MNIST filters look at the margins of digits and stroke outlines.

- Tiny-ImageNet filters can see the borders of things, color gradients, and texture.

This indicates that CNNs learn characteristics that are important to space and fit the dataset's level of difficulty.

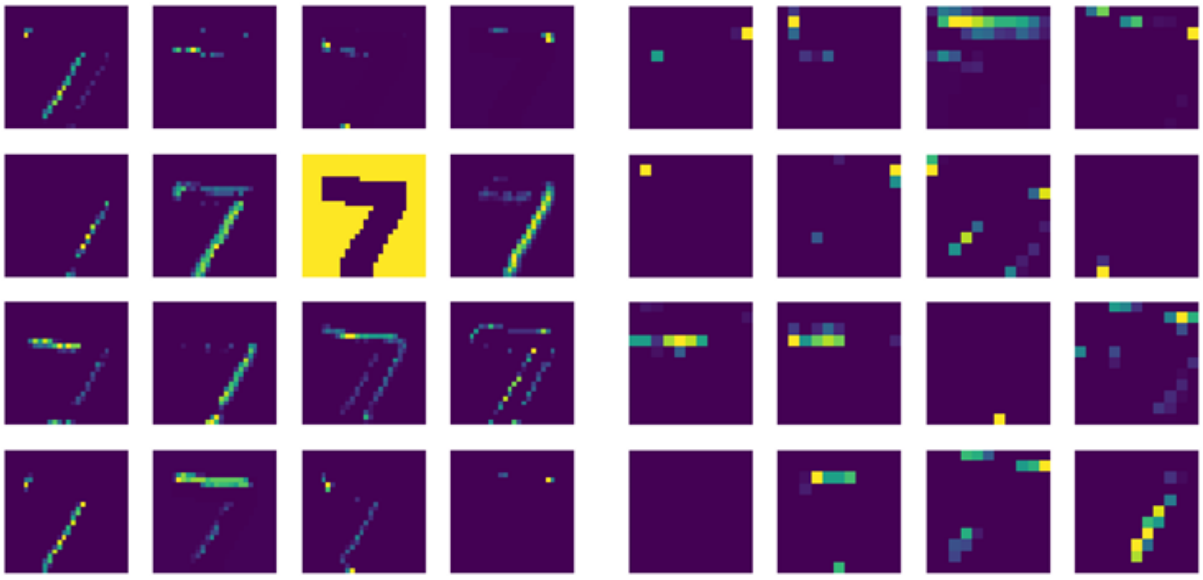
0.1 3.2 The Receptive Field Experiment

Goal

The aim of this experiment is to investigate the progression of feature abstraction over varying network depths by presenting activation maps following:

- The first layer of convolution
- The last layer of convolution

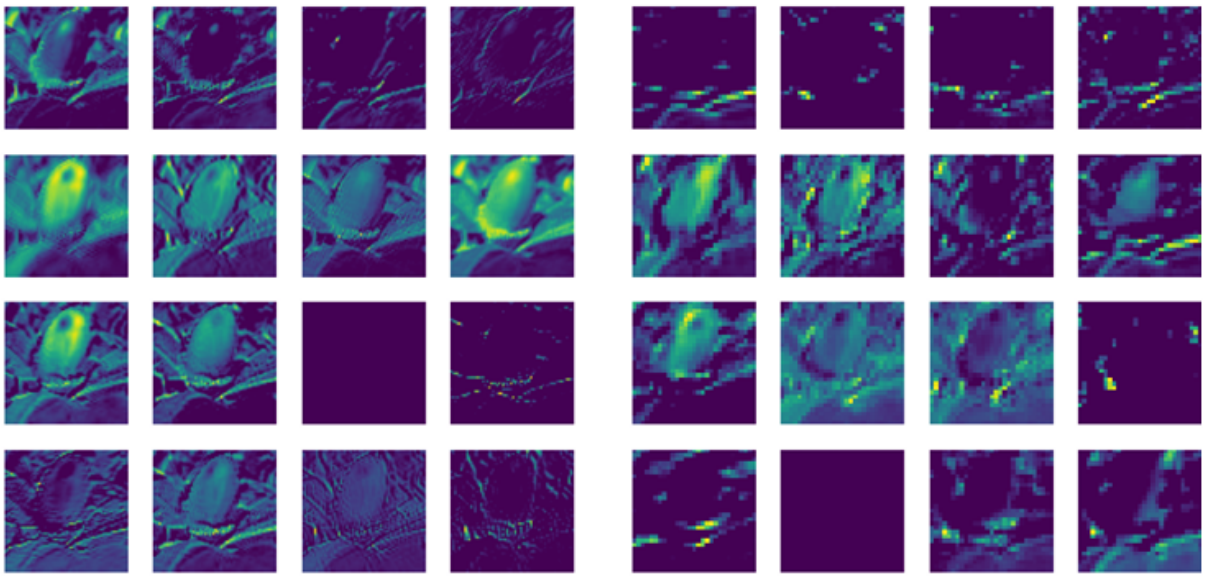
MNIST Activation Maps



(a) After First Convolution (b) After the Last Convolution

Figure 3: A side-by-side comparison of the MNIST activation maps from the first and last convolutional layers.

Tiny-ImageNet-10 Activation Maps



(a) After the First Convolution (b) After the Last Convolution

Figure 4: A comparison of the activation maps for Tiny-ImageNet-10 between the first and last layers of convolution.

Interpretation

CNNs learn how to show features in a hierarchy:

1. The initial layers look for basic things like edges and gradients.
2. Intermediate layers put these together to build shapes and parts.
3. Abstractions at the object level are stored at deeper levels.

The steady growth of the receptive field makes it easier to go from finding local features to understanding global objects.

This hierarchical abstraction is a big part of why CNNs fare better than fully connected networks when it comes to vision tasks.

The End of Part 3

The filters and activation maps show that CNNs don't work like black boxes. Instead, they develop structured, hierarchical representations that range from simple edge detection to more abstract, high-level concepts.

When you go from MNIST to Tiny-ImageNet-10, the learnt features get a lot more complicated. This is because the dataset offers a greater range of styles for how things look.

Part-4: Advanced Optimization Robustness

Part-4.1: The Depth vs. Normalization Duel

1 Goal

Our objective is to study the effect of Batch Normalization on the training stability, convergence behavior, and final performance of a Convolutional Neural Network(CNN) for image classification.

2 The Task

Our task is to train and compare two models on a 10-class Tiny-ImageNet subset where models are,

- **Model A (Baseline CNN)**: CNN without Batch normalization
- **Model B (CNN+BatchNorm)**: Same CNN architecture with Batch normalization layers

In addition to accuracy comparison, we also analyze the activation statistics (mean and variance) of an intermediate layer to understand how Batch Normalization stabilizes training.

- Dataset:Tiny-imageNet subset(10-classes)
- Image size: $128 \times 128 \times 3$
- Split:
  - Training set: 3500 images
  - Validation set: 500 images
  - Test set: 1000 images
- Task: Multiclass image classification (10 classes)

All images were normalized to the range $[0, 1]$ by dividing pixel values by 255.

3 Network Architectures

Model A:CNN without batch normalization The baseline CNN uses a stack of convolutional blocks followed by Global Average Pooling(GAP) and dense layers:

- Input: $128 \times 128 \times 3$ image
- Block 1:
  - Conv2D (12 filters,3×3,ReLU activation)
  - Conv2D (12 filters, 3×3,ReLU activation)

  - MaxPooling2D (2×2,stride=2)

- Block 2:
  - Conv2D (24 filters,3×3,ReLU)
  - Conv2D (24 filters,3×3,ReLU)
  - MaxPooling2D (2×2,stride=2)

- Block 3:
  - Conv2D (48 filters, 3×3,ReLU)
  - Conv2D (48 filters, 3×3,ReLU)

- GlobalAveragePooling2D
- Dense (32 neuron,ReLU)
- Output: Dense (10 neuron,softmax)

Model B: CNN with batch normalization and dropout This model uses the same conditions as model A, but with the following improvements :

- Batch normalization after each convolution layer
- ReLU activation applied after Batch normalization
- Dropout layers added after pooling and dense layers to reduce overfitting

4 Training Details

Model A and B both were trained under the following conditions:

- Optimizer:Adam
- Loss Function:Sparse categorical crossentropy
- Batch Size:32
- Input size:$128 \times 128 \times 3$
- Callbacks:Earlystopping (patience = 10–12, restore best weights for Model B)
- No. of epoch:
  - Model A:50 epoch
  - Model B:55 epoch

5 Performance comparison

The final evaluation results on the test set are

Model	Test accuracy	Test loss
Model A (Without BN)	60.1%	1.152
Model B (With BN)	66.9%	0.963

Table 1: Test performance comparison between baseline CNN and CNN with Batch normalization.

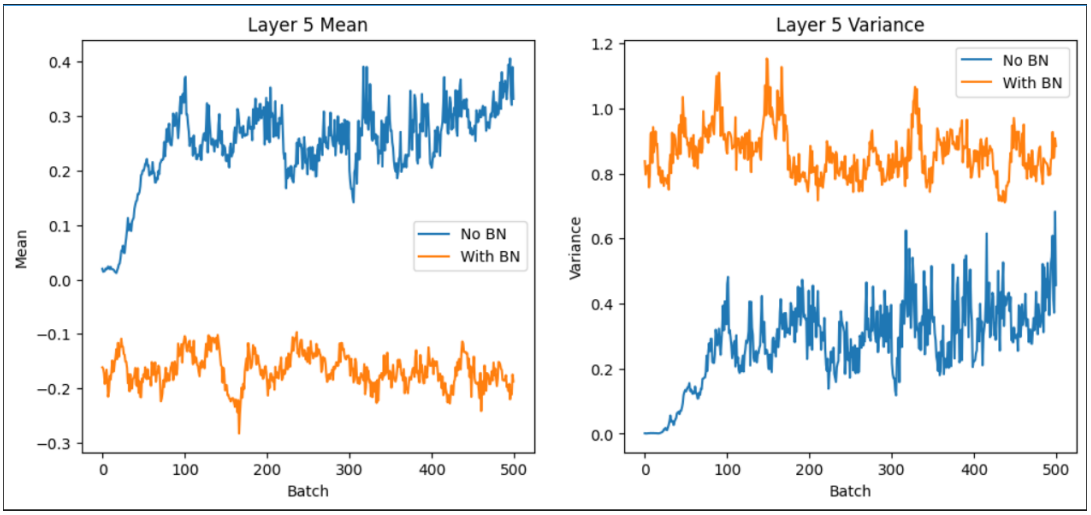


Figure 1: Mean and variance of activations at layer-5 over training batches for the CNN without batch normalization and with batch normalization.

6 Activation Statistics Analysis (Layer 5)

To further analyze the effect of Batch Normalization, the mean and variance of activations from an intermediate layer ('layer-5') were observed over 500 batches during training.

Observations

Without batch normalization:

- The mean of activations drifts significantly over time.
- The variance increases and fluctuates heavily across batches.
- This indicates internal covariate shift, making optimization harder and less stable.

With batch normalization:

- The mean of the activations stay more stable and does not change too much.
- The variance of the activations also stay within a limited and controlled range.
- This showing that batch normalization keeps the layer inputs well-scaled, which makes the training more stable and predictable.

7 Why does batch normalization Helpful

Batch normalization improve the performance and training stability due to following reasons:

- Reduce internal covariate shift:By normalizing activations within each batch and distribution of inputs to each layer remains more stable throughout training.
- Faster and more stable convergence:Stable activation distributions allow the optimizer to converge faster.
- Regularization effect:The noise introduced by batch statistics acts as a form of regularization and reducing overfitting.
- Better gradient flow:Normalized activations prevent exploding or vanishing activations and improve gradient propagation.

8 Conclusion

This experiment demonstrates the benefit of Batch normalization in CNN training:

- The model A achieved higher test accuracy of 66.9
- Model A has lower test loss which indicating a better generalization.
- Activation statistics show that batch normalization stabilizes both mean and variance across batches.
- Overall, batch normalization leads to more stable training, faster convergence,and better performance.

Part 4.2: Data Augmentation "Sanity Check"

1 Goal

Our goal is to study the effect of data augmentation on the performance of a Convolutional Neural Network (CNN)

2 The Task

The task is to perform multi-class image classification on a 10-class subset of the Tiny-ImageNet dataset. Each input image has a size of $128 \times 128 \times 3$, and the network must predict the correct class label among 10 possible classes. Two models with the same architecture are trained:

- **Model A (without augmentation):** Trained on the original training images only.
- **Model B (with augmentation):** Trained using randomly augmented images where agumentation is done by random rotation,brightness,contrast.

The performance of both models is compared using training accuracy and test accuracy to evaluate whether data augmentation improves generalization.

3 Dataset and Preprocessing

- Dataset: Tiny-ImageNet subset (10 classes)
- Image size: $128 \times 128 \times 3$
- Batch size: 32
- Data split: Training, Validation, and Test sets

All images are rescaled to the range $[0, 1]$ by dividing pixel values by 255.
For the augmented model, the following transformations are applied during training:

- Random Rotation
- Random Brightness
- Random Contrast

These transformations were used to generate the different version of same image to increase diversity of the training data.

4 Model Architecture

Both models use the same CNN architecture consisting of:

- Multiple convolutional layers with ReLU activation
- Batch normalization layers
- MaxPooling layers
- Dropout layers for regularization
- Global Average Pooling
- A fully connected layer with ReLU activation
- An output layer with softmax activation for 10-class classification

Since both models use the same network and training settings, the only difference between them is the use of data augmentation. Therefore,if any change in performance are only due to data augmentation.

5 Training Details

Both models were trained with the following conditions:

- Optimizer: Adam
- Loss Function: Sparse categorical crossentropy
- Batch size: 32
- Epochs: 50
- Callback: Earlystopping(patience = 10)

6 Results

The final training and test accuracies are :

Model	Training accuracy	Test accuracy
Without Augmentation	66.6%	60.8%
With Augmentation	68.1%	63.1%

Table 1: Performance of both models trained with and without data augmentation

7 Analysis

From the results, it can be observed that the model trained with data augmentation achieves higher test accuracy than the model trained without augmentation. The training accuracy goes up only a little, but the test accuracy goes up a lot more, which shows that the model is better at generalizing.

Data augmentation helps the model to see more different variety of the same images, making it less sensitive to small changes such as rotation, brightness, and contrast in the image. This reduces overfitting and forces the model to learn more robust features.

Data augmentation is more useful for the test set than for the training set. The accuracy of the training set goes up only a little, but the accuracy of the test set goes up a lot. This is because augmentation is mostly a way to regularize the model: it makes the training data more varied and stops the model from overfitting. Because of this, the model works better on images it never seen before.

8 Conclusion

This experiment confirms that data augmentation is an effective way to improve generalization in CNN:

- The model with augmentation achieves higher test accuracy than the model without augmentation.
- Augmentation reduces overfitting by exposing the network to more diverse training samples.
- Even simple transformations such as rotation, brightness, and contrast changes can lead to more robust and better-performing models.